



统计计算大作业报告

贝叶斯网络结构学习中的 MCMC 方法： 算法设计、比较与实验分析

课程：统计计算

姓名：黄至刚 黄梦凯 黄旻皓 贾耀杰

学号：2313915 2312197 2212158 2212033

学院：统计与数据科学学院

学期：2026 Spring

南开大学

2026 年 6 月 18 日

摘 要

贝叶斯网络结构学习旨在从观测数据中推断变量之间的有向无环依赖关系。随着节点数增加, DAG 空间呈超指数增长, 同时每个候选结构必须满足无环约束, 使得精确结构学习在计算上面临较高复杂度。本文从算法设计动机出发, 系统分析 Structure MCMC、Order MCMC 和 Partition MCMC 三类采样策略在状态空间构造、proposal 机制和混合效率方面的核心差异。在此基础上, 本文加入三项扩展分析模块: 自适应父节点集约束、边后验熵不确定性度量以及多链诊断框架。实验部分构建了从数据模拟、自主实现到 BiDAG 官方包复现的完整链路, 并在低维和中等规模设置下进行比较, 同时补充 iterative 策略的规模扩展实验。代码层面, 实验框架通过 checkpoint 机制支持断点续跑与结果追踪。结果表明, Order MCMC 和 Partition MCMC 通过重构状态空间缓解了直接 DAG 搜索的混合困难; 结合预筛选与迭代修正的 hybrid/iterative 策略可作为较大规模网络上的实用补充。边后验不确定性分析进一步揭示了不同方法在结构置信度量化方面的差异, 为实际应用中的方法选择提供参考。

关键词: 贝叶斯网络; 结构学习; MCMC; Order MCMC; Partition MCMC; BiDAG; 状态空间设计; 边后验概率; 收敛诊断

目录

1	引言	1
1.1	问题背景与研究动机	1
1.2	研究问题	1
1.3	本文贡献	2
1.4	论文组织	2
2	理论基础	3
2.1	贝叶斯网络的概率语义	3
2.2	结构后验与边际似然	3
2.3	BGe 评分与可分解性	3
2.4	自主实现中的 Gaussian BIC 评分	4
2.5	边后验概率与结构不确定性	4
2.6	DAG 空间的组合规模	4
3	MCMC 方法的算法设计与动机	5
3.1	Structure MCMC: 基准与局限	5
3.1.1	设计动机	5
3.1.2	状态空间与 Proposal	5
3.1.3	算法伪代码	6
3.1.4	核心问题分析	7
3.2	Order MCMC: 状态空间重构	7
3.2.1	设计动机	7
3.2.2	状态空间、父集枚举与评分	8
3.2.3	Proposal 与 MH 接受率	8
3.2.4	排序偏差问题	8
3.2.5	机制图	10
3.3	Partition MCMC: 缓解排序偏差	10
3.3.1	设计动机	10
3.3.2	完整 Partition MCMC 的核心要素	11
3.3.3	对照实现	11
3.3.4	三种状态空间的统一视角	11
3.4	Hybrid / Iterative MCMC: 规模扩展策略	11
3.4.1	设计动机	11
3.4.2	算法流程	13
3.5	方法比较总结	13

4 扩展分析模块	15
4.1 自适应父节点集约束策略	15
4.1.1 动机	15
4.1.2 方法	15
4.1.3 实验结果	16
4.2 基于边后验熵的结构不确定性量化	16
4.2.1 动机	16
4.2.2 方法	16
4.2.3 实验结果	16
4.3 多链收敛诊断方案	17
4.3.1 动机	17
4.3.2 方法	17
5 实验设计	18
5.1 数据生成机制	18
5.2 评价指标	18
5.3 实验矩阵	18
5.4 实验基础设施	18
6 实验结果与分析	20
6.1 初步运行检查：最小链路验证	20
6.2 自主实现验证	20
6.3 自主实现与 BiDAG 的结果对比	21
6.4 低维 Order 与 Partition 比较	23
6.5 中等规模可扩展性	24
6.6 Iterative MCMC 规模扩展补充实验	24
6.7 边后验不确定性分析	24
6.8 样本量影响	25
6.9 父节点集约束敏感性	26
6.10 中等规模关键论文趋势对照实验	28
7 讨论	33
7.1 状态空间设计	33
7.2 规模扩展实验的定位	33
7.3 不确定性量化	33
7.4 局限与展望	33
8 结论	35

A 实验环境与可复现性	37
A.1 软件环境	37
A.2 代码结构	37
A.3 实验复现	37

1 引言

1.1 问题背景与研究动机

贝叶斯网络 (Bayesian network, BN) 是一类用有向无环图 (directed acyclic graph, DAG) 编码变量间条件独立关系的概率图模型。图中的每个节点对应一个随机变量，有向边 $i \rightarrow j$ 表示变量 X_i 对 X_j 存在直接的条件依赖关系。贝叶斯网络的核心优势在于其将高维联合分布分解为局部条件分布的乘积，使复杂的依赖结构具有清晰的可解释性。

在生物信息学、流行病学、社会科学和因果推断等应用中，研究者不仅关心模型的预测精度，更关心变量之间的结构关系本身——即哪些变量之间存在直接的调控或影响路径。从观测数据中学习这样的网络结构，被称为贝叶斯网络结构学习 (Bayesian network structure learning) [4]。

结构学习的根本困难来自组合爆炸。Chickering (1996) 给出了从数据中学习最优贝叶斯网络结构的 NP-hard 结果 [1]；关于贝叶斯网络结构学习方法的系统综述可参考 Kitson et al. (2023)[5]。对于一个包含 p 个节点的有向无环图，可能的 DAG 数量 a_p 满足递推关系：

$$a_p = \sum_{k=1}^p (-1)^{k-1} \binom{p}{k} 2^{k(p-k)} a_{p-k}, \quad (1)$$

其中 $a_0 = 1$ 。该数列呈超指数增长：当 $p = 3$ 时仅有 25 个 DAG， $p = 5$ 时增至约 2.9×10^4 ， $p = 10$ 时已超过 4.2×10^{18} 。这一组合复杂度意味着除极低维问题外，枚举所有可能结构在计算上并不可行。

与此同时，无环约束为结构搜索带来了额外的计算负担。在局部搜索过程中，每当尝试添加、删除或反转一条边，都需要验证结果图是否仍为 DAG。对于大规模网络，这一验证步骤本身就需要 $O(p + |E|)$ 的时间复杂度。

面对这些挑战，基于 Markov chain Monte Carlo (MCMC) 的贝叶斯方法提供了一条可行路径：不只寻找单一最优结构，而是从结构后验分布中采样，通过样本估计边后验概率，从而量化结构不确定性。然而，如何在庞大的 DAG 空间中设计能够高效混合的 MCMC 采样器，本身就是一个值得研究的问题。

1.2 研究问题

本文关注以下四个问题：

问题一：状态空间设计。直接以 DAG 为状态空间的 Structure MCMC 面临哪些困难？Order MCMC 和 Partition MCMC 如何通过重构状态空间来应对这些困难，又引入了哪些新问题？

问题二：规模扩展性。当节点数增加时，普通 MCMC 方法的计算压力如何变化？hybrid/iterative 策略通过缩小候选搜索空间能在多大程度上缓解这一压力？

问题三：不确定性量化。 MCMC 采样如何超越 MAP 点估计来提供结构不确定性的量化？不同的状态空间表示对边后验概率的估计有何影响？

问题四：扩展分析。 能否通过自适应策略、收敛诊断等模块提升实验框架的诊断能力和实用性？

1.3 本文贡献

针对上述问题，本文的主要贡献包括：

1. **算法动机分析。** 从状态空间几何、proposal 效率和混合时间等角度，分析了每种 MCMC 策略的设计动机和内在逻辑。
2. **自主 MCMC 实现。** 在 R 语言中构建了 Structure MCMC、Order MCMC 和 Partition MCMC 对照采样器，覆盖评分计算、proposal 生成、MH 接受和边后验概率估计的完整链路，用于阐明算法的计算机制。
3. **BiDAG 官方包的系统复现。** 基于 BiDAG 包构建了统一的实验框架，实现了 Order MCMC、Partition MCMC 和 iterative MCMC 的可复现比较。
4. **三项扩展分析模块。** 加入自适应父节点集约束、基于边后验熵的不确定性量化以及多链收敛诊断方案。
5. **可复现的实验框架。** 实验包含 checkpoint 机制、日志记录和结果追踪，确保主要表格与图像可由脚本复现。

1.4 论文组织

第 2 节给出贝叶斯网络结构学习的理论基础，第 3 节阐述各 MCMC 方法的算法设计动机与实现细节，第 4 节介绍文中加入的扩展分析模块，第 5 节描述实验设计，第 6 节报告实验结果与分析，第 7 节进行讨论，第 8 节总结全文。附录 A 说明实验环境与可复现性。

2 理论基础

2.1 贝叶斯网络的概率语义

设 $\mathcal{G} = (V, E)$ 是节点集 $V = \{1, 2, \dots, p\}$ 上的有向无环图。记 $\mathbf{X} = (X_1, \dots, X_p)$ 为 p 维随机向量。贝叶斯网络假设联合概率密度（或质量函数）可分解为每个变量以其父节点为条件的局部分布的乘积：

$$P(\mathbf{X} | \mathcal{G}, \Theta) = \prod_{j=1}^p P(X_j | \mathbf{X}_{\text{Pa}_{\mathcal{G}}(j)}, \Theta_j), \quad (2)$$

其中 $\text{Pa}_{\mathcal{G}}(j) = \{i \in V : i \rightarrow j \in E\}$ 表示节点 j 在图 $\mathcal{G}$ 中的父节点集合， $\Theta = (\Theta_1, \dots, \Theta_p)$ 为所有局部条件分布的参数。

这一分解具有两个重要含义。第一，它将 p 维联合分布的建模问题转化为 p 个低维条件分布的建模问题，从而降低参数空间的维度。第二，图中的缺失边直接对应条件独立断言：若 i 不是 j 的父节点（且不存在间接路径使得 i 成为 j 的祖先），则在给定 j 的父节点后， X_i 与 X_j 条件独立。

2.2 结构后验与边际似然

给定观测数据 $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}\}$ ，贝叶斯结构学习的目标是推断图结构 $\mathcal{G}$ 的后验分布：

$$P(\mathcal{G} | \mathcal{D}) = \frac{P(\mathcal{D} | \mathcal{G})P(\mathcal{G})}{\sum_{\mathcal{G}'} P(\mathcal{D} | \mathcal{G}')P(\mathcal{G}')}, \quad (3)$$

其中 $P(\mathcal{D} | \mathcal{G})$ 是边际似然（marginal likelihood），通过对参数 Θ 积分得到：

$$P(\mathcal{D} | \mathcal{G}) = \int P(\mathcal{D} | \mathcal{G}, \Theta)P(\Theta | \mathcal{G}) d\Theta. \quad (4)$$

边际似然的计算是贝叶斯结构学习的核心计算挑战之一。对于离散数据和多项分布模型，若参数先验取共轭的 Dirichlet 分布，可得到封闭形式的 BDe (Bayesian Dirichlet equivalence) 评分。对于连续数据和高斯线性模型，若采用 Normal-Wishart 共轭先验，可得到 BGe (Bayesian Gaussian equivalence) 评分。

2.3 BGe 评分与可分解性

本文实验中的 BiDAG 复现统一采用 BGe 评分。BGe 评分的关键性质是可分解性 (decomposability) [4]：整张图的评分可分解为各节点局部评分的和：

$$S(\mathcal{G}; \mathcal{D}) = \log P(\mathcal{D} | \mathcal{G}) = \sum_{j=1}^p s_j(\text{Pa}_{\mathcal{G}}(j); \mathcal{D}). \quad (5)$$

可分解性对 MCMC 至关重要。当 MCMC proposal 只改变少数边（因而只改变少数节点的父集）时，全局评分的更新只需重新计算受影响节点的局部评分，而不必重新评估整张图。这一性质将每次 MCMC 迭代的计算代价从 $O(p)$ 降至 $O(|\text{affected nodes}|)$ 。

2.4 自主实现中的 Gaussian BIC 评分

本文自主实现采用 Gaussian BIC 型评分。对于节点 j ，给定父集 Pa_j ，考虑线性回归模型：

$$X_j = \beta_{j0} + \sum_{i \in \text{Pa}_j} \beta_{ji} X_i + \varepsilon_j, \quad \varepsilon_j \sim \mathcal{N}(0, \sigma_j^2). \quad (6)$$

记残差平方和为 RSS_j ，则局部对数似然在 MLE 处的值为

$$\ell_j(\hat{\Theta}_j) = -\frac{n}{2} \left[\log(2\pi) + 1 + \log \left(\frac{\text{RSS}_j}{n} \right) \right]. \quad (7)$$

局部 BIC 型得分为

$$\text{BIC}_j(\text{Pa}_j) = \ell_j(\hat{\Theta}_j) - \frac{d_j}{2} \log n, \quad (8)$$

其中 $d_j = |\text{Pa}_j| + 2$ 是局部模型的参数个数（包括截距和误差方差）。BIC 并非完整贝叶斯边际似然，但在 Gaussian 线性模型下可作为 BGe 评分的渐近近似，用于构建可审计的对照采样器并分析评分函数对 MCMC 搜索行为的影响。

2.5 边后验概率与结构不确定性

MCMC 采样的一个重要优势是能够估计边后验概率，从而量化结构的不确定性。设 burn-in 后保留了 M 个图样本 $\mathcal{G}^{(1)}, \dots, \mathcal{G}^{(M)}$ ，定义边 $i \rightarrow j$ 的后验包含概率为

$$\hat{P}(i \rightarrow j \mid \mathcal{D}) = \frac{1}{M} \sum_{m=1}^M \mathbf{1}\{i \rightarrow j \in \mathcal{G}^{(m)}\}. \quad (9)$$

边后验概率提供了比单一 MAP 图更丰富的信息。当 $\hat{P}(i \rightarrow j \mid \mathcal{D}) \approx 0$ 或 ≈ 1 时，数据对该边的存在性给出了较强信号；当该概率接近中间值（如 0.3–0.7）时，说明有限数据和有限采样下仍存在结构歧义。本文在第 4 节中将进一步引入基于边后验熵的不确定性量化框架。

2.6 DAG 空间的组合规模

理解 MCMC 方法设计的动机，需要首先认识 DAG 空间的组合规模。表 1 给出了不同节点数下的 DAG 数量，说明枚举方法为何不可行。当 $p \geq 10$ 时，DAG 数量已超出常规枚举范围。即使采用 MCMC，巨大的状态空间也对 proposal 设计和链长提出了极高要求。

表 1: DAG 数量随节点数的增长

节点数 p	DAG 数量 (近似)	数量级
3	2.5×10^1	可枚举
5	2.9×10^4	可枚举
8	1.1×10^{11}	不可枚举
10	4.2×10^{18}	不可枚举
15	1.1×10^{41}	超出常规枚举范围
20	2.3×10^{72}	天文数字

3 MCMC 方法的算法设计与动机

本节从计算瓶颈出发, 阐述各类 MCMC 策略的设计动机, 分析其状态空间构造与前置方法局限之间的关系, 并以统一的数学符号给出形式化描述。

3.1 Structure MCMC: 基准与局限

3.1.1 设计动机

一种自然方案是在 DAG 空间本身定义马尔可夫链。状态 $\mathcal{G}$ 为一张 DAG, proposal 通过局部边操作在 DAG 空间中移动。该方法的目标分布直接定义在 DAG 后验空间上, 因此采样状态与推断目标一致, 无需额外的编码和解码。

然而, 该方案存在三个方面的困难。其一, DAG 空间的结构较不规则: 高后验概率区域之间常被低概率区域分隔, 而每次仅改变一条边的局部操作难以跨越这些区域。其二, 无环约束降低了 proposal 的有效接受率: 随机 add 或 reverse 操作有较大概率产生有向环而被拒绝。其三, DAG 空间中的编辑距离与后验相似性并不一致, 两张 SHD 相近的图可能具有差异较大的后验概率。这些因素共同导致 Structure MCMC 在节点数增加后出现混合困难。针对局部边操作混合效率不足的问题, Grzegorzcyk and Husmeier (2008) 提出了改进的 edge reversal move, 为后续 proposal 设计提供了重要参考 [3]。

3.1.2 状态空间与 Proposal

Structure MCMC 定义三种基本 proposal 操作:

- **Add edge:** 在非边位置 (i, j) 添加有向边 $i \rightarrow j$ 。
- **Delete edge:** 删除一条已有边 $i \rightarrow j$ 。
- **Reverse edge:** 将已有边 $i \rightarrow j$ 替换为反向边 $j \rightarrow i$ 。

每次 proposal 后需要检查两个约束: (1) 结果图是否为 DAG (可以通过拓扑排序或深度优先搜索检验); (2) 是否满足最大父节点数限制 (防止单个节点的父集过大, 控制局部评分计算的复杂度)。

3.1.3 算法伪代码

算法 1 Structure MCMC 单步迭代

Require: 当前状态 G (邻接矩阵), 局部评分表 S , 最大父节点数 K

Ensure: 更新后的状态 G , 接受标志

```

1  随机选择移动类型  $m \in \{\text{add}, \text{delete}, \text{reverse}\}$ 
2   $G' \leftarrow G$ 
3  if  $m = \text{add}$  then
4      随机选取非边  $(i, j)$  且  $|\text{Pa}_{G'}(j)| < K$ 
5       $G'_{ij} \leftarrow 1$ 
6  else if  $m = \text{delete}$  then
7      随机选取已有边  $(i, j)$ 
8       $G'_{ij} \leftarrow 0$ 
9  else ▷ reverse
10     随机选取已有边  $(i, j)$  且  $|\text{Pa}_{G'}(i)| < K$ 
11      $G'_{ij} \leftarrow 0; G'_{ji} \leftarrow 1$ 
12 end if
13 if  $\neg \text{IsDAG}(G')$  then
14     return  $G$ , rejected ▷ DAG 约束失败
15 end if
16  $\Delta \leftarrow \text{SCORE}(G', S) - \text{SCORE}(G, S)$ 
17 if  $\log(\text{Uniform}(0, 1)) < \Delta$  then
18     return  $G'$ , accepted
19 else
20     return  $G$ , rejected
21 end if

```

3.1.4 核心问题分析

Structure MCMC 的状态定义较为直接, 但在计算上存在三个主要问题:

(a) **移动效率低**。在稀疏图中, 大多数 (i, j) 对是非边, add proposal 随机采样非边时命中高概率边的概率极低。delete proposal 面临对称困难。reverse proposal 需要目标位置有父节点容量。总体而言, 每次局部移动只能改变图中极小的局部结构。

(b) **无环检查频繁且昂贵**。每步都需要判定候选图是否为 DAG。对于大图, 一次拓扑排序或 DFS 检查需要 $O(p + |E|)$ 时间, 当 p 较大且链较长时, 这部分开销占总计算时间的不可忽略比例。

(c) **混合缓慢**。DAG 空间高度多峰且存在低概率分隔区域。局部边操作难以在有限步数内跨越不同的高概率区域, 导致链在局部模式附近长时间徘徊。

Structure MCMC 的单步迭代流程为：从当前图 G 出发，随机选择加边、删边或反转边中的一种操作生成候选图 G' ，依次通过无环检查和 MH 接受准则两重筛选（含环则直接拒绝），burn-in 后将每次迭代的当前图累积到边后验计数中（见图 1）。

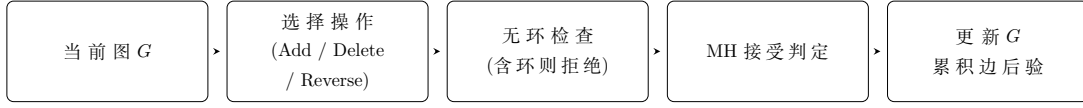


图 1: Structure MCMC 单步迭代流程

3.2 Order MCMC: 状态空间重构

3.2.1 设计动机

Structure MCMC 的主要困难源于在 DAG 空间中直接进行局部操作。每次 proposal 后都需要检查无环性，这不仅带来 $O(p + |E|)$ 的额外计算开销，也会使大量随机 proposal 因产生有向环而被拒绝，从而降低有效移动比例。

Order MCMC (Friedman and Koller, 2003) [2] 的核心思路是：**通过改变状态空间来处理无环约束**。该方法将状态从 DAG 重构为变量的全序排列 $\prec$ ，并规定边只能从排序靠前的节点指向排序靠后的节点。在这一约定下，任何根据排序生成的图都自动无环，因为有向环要求至少存在一条从排序后方指向前方的边，而排序约束排除了该情形。

这一设计的基本思想是**用表示约束替代验证约束**：将状态定义在天然满足无环约束的表示空间中，而不是在每次 proposal 后验证候选图是否合法。Order MCMC 的代价是状态空间扩大， $p!$ 个排列远多于实际存在的 DAG 数量；但由于每个排列均可较快评估，且排列间移动仅涉及随机交换两个位置，整体采样效率通常得到改善。

具体地，设排列 $\prec = (v_1, v_2, \dots, v_p)$ ，则边 $v_a \rightarrow v_b$ 只有在 $a < b$ 时才被允许。这一定义自动保证无环性，不需要显式检查。

3.2.2 状态空间、父集枚举与评分

给定一个排列 $\prec$ ，节点 j 的候选父集被限定为所有排在 j 之前的节点的子集：

$$\mathcal{P}(j | \prec) = \{S \subseteq \{v_1, \dots, v_{k-1}\} : |S| \leq K\}, \quad (10)$$

其中 k 是节点 j 在 $\prec$ 中的位置。这一约束使每个节点的父集搜索空间从 2^{p-1} 降至 2^{k-1} ——虽然上界仍是指数，但对于靠前位置的节点，候选空间极小。

给定排列后，可以为每个节点确定最优父集（使局部评分最大化）：

$$\text{Pa}_j^*(\prec) = \arg \max_{S \subseteq \{v_1, \dots, v_{k-1}\}, |S| \leq K} s_j(S; \mathcal{D}). \quad (11)$$

这可以通过枚举或启发式搜索完成。排列 $\prec$ 的总评分为各节点最优得分的和：

$$S(\prec; \mathcal{D}) = \sum_{j=1}^p s_j(\text{Pa}_j^*(\prec); \mathcal{D}). \quad (12)$$

3.2.3 Proposal 与 MH 接受率

Order MCMC 的 proposal 采用随机 swap 操作：随机选择两个位置并交换。设当前排列为 $\prec$ ，proposal $\prec'$ 通过交换两个随机位置 a 和 b 得到：

$$\prec' = (v_1, \dots, v_{a-1}, v_b, v_{a+1}, \dots, v_{b-1}, v_a, v_{b+1}, \dots, v_p). \quad (13)$$

这是一种对称 proposal，因此 MH 接受率简化为

$$\alpha(\prec \rightarrow \prec') = \min \left\{ 1, \frac{P(\mathcal{D} | \prec') P(\prec')}{P(\mathcal{D} | \prec) P(\prec)} \right\} \approx \min \{1, \exp(S(\prec'; \mathcal{D}) - S(\prec; \mathcal{D}))\}, \quad (14)$$

其中后一步近似假设排列先验均匀。

算法 2 Order MCMC 单步迭代

Require: 当前排列 $\prec$ ，局部评分表 S_{local} ，最大父节点数 K

Ensure: 更新后的排列 $\prec$ ，对应的 DAG $G_{\prec}$

```

1  随机选择两个位置  $a, b \in \{1, \dots, p\}$ ,  $a \neq b$ 
2   $\prec' \leftarrow \prec$  交换位置  $a$  和  $b$  后的排列
3   $S_{\text{new}} \leftarrow 0$ 
4  for each node  $j$  in  $\prec'$  do
5       $\text{Pa}_j^* \leftarrow \arg \max_{S \subseteq \text{predecessors}(j, \prec'), |S| \leq K} s_j(S)$ 
6       $S_{\text{new}} \leftarrow S_{\text{new}} + s_j(\text{Pa}_j^*)$ 
7  end for
8   $\Delta \leftarrow S_{\text{new}} - S_{\text{old}}$ 
9  if  $\log(\text{Uniform}(0, 1)) < \Delta$  then
10      $\prec \leftarrow \prec'$ ,  $S_{\text{old}} \leftarrow S_{\text{new}}$ 
11 end if
12 if 在 burn-in 后 then
13      $G_{\prec} \leftarrow$  从  $\prec$  确定的约束中采样父集
14     累积边计数  $E \leftarrow E + G_{\prec}$ 
15 end if
```

3.2.4 排序偏差问题

虽然 Order MCMC 通过状态空间重构避免了 DAG 检查，但它引入了一个新的偏差来源。同一个 DAG 可以对应多个不同的排列：只要父节点排在子节点之前的排列都是该 DAG 的合法排列。因此：

$$P(\mathcal{G} | \mathcal{D}) = \sum_{\prec: \mathcal{G} \text{ compatible with } \prec} P(\prec | \mathcal{D}). \quad (15)$$

如果直接在排列空间均匀采样并取 MCMC 中 DAG 样本的频率，那么对应于更多排列的 DAG 会被过度表示。这种“排序空间膨胀”效应被称为 Order MCMC 的排序偏差 (order bias)。

3.2.5 机制图

Structure MCMC 与 Order MCMC 的核心差异在于状态空间的选择（见图 2）：前者直接在 DAG 空间操作，每步需显式检查无环，proposal 因产生环而被拒绝的概率较高；后者将状态重构为排列以自动保证无环，proposal 采用随机 swap，但需在每次 swap 后重算父集得分，且引入了同一 DAG 对应多个排列的排序偏差。

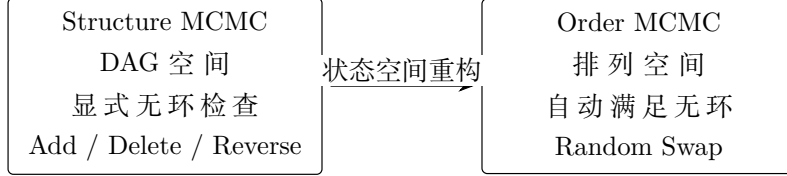


图 2: Structure MCMC 与 Order MCMC 的对比

3.3 Partition MCMC: 缓解排序偏差

3.3.1 设计动机

Order MCMC 以排列空间替代 DAG 空间，避免了逐步无环检查，但引入了**排序偏差** (order bias)：同一 DAG 通常兼容于多个排列（父节点只需排在子节点之前即可），在排列空间中均匀随机游走会使兼容更多排列的 DAG 获得更高的采样频率，从而影响后验估计。

Partition MCMC (Kuipers and Moffa, 2017) [6] 针对上述排序偏差提出。其核心思想是：**用有序 partition 作为比全序更粗、比 DAG 更规整的状态表示**。该方法将节点划分为有序的块，同块节点间允许任意 DAG 结构，块间边只能从前块指向后块。因此，排列中由同块内部不同排序产生的冗余被合并，一个有序 partition 对应的 DAG 集合比单个排列对应的 DAG 集合更紧凑，从而减小表示偏差。

三种状态空间形成了清晰的层级关系：DAG 空间是最精确的表示，但最难搜索；排列空间 ($p!$ 个状态) 是最宽松的表示，容易搜索但偏差最大；有序 partition 空间介于两者之间——它比 DAG 空间更规整（块间单向约束天然保证无环），比排列空间更精确（减少了同一 DAG 的多重计数）。

形式化地，一个有序 partition 将节点集合划分为 B 个有序的块：

$$\Pi = (\Pi_1, \Pi_2, \dots, \Pi_B), \quad (16)$$

块间互不相交且覆盖所有节点，边只能从前块指向后块（同块内可有任意 DAG 结构）。当 $B = p$ 时退化为全序 (Order MCMC)，当 $B = 1$ 时退化为无约束 DAG (Structure MCMC)。有序 partition 因此是前两种状态空间的自然推广：通过调节块的数量和粒度，可以在表示精度和搜索便利性之间进行权衡。

3.3.2 完整 Partition MCMC 的核心要素

本文依赖 BiDAG 实现完整的 Partition MCMC (Kuipers and Moffa, 2017)。为理解其机制, 需要认识该方法的三个核心要素:

要素一: DAG 到 Partition 的组合计数。给定一个有序 partition Π , 需计算兼容于 Π 的 DAG 数量 (或权重), 确保 MCMC 在 partition 空间的平稳分布正比于所有兼容 DAG 的后验概率之和。这涉及同块内部 DAG 结构的组合计数, 是算法中最复杂的部分。

要素二: 基于权重的 proposal。Partition MCMC 的 proposal 包括 split (拆分一个块)、merge (合并相邻块) 和 shift (将节点移至相邻块)。这些 proposal 的接受率需要包含 DAG 权重因子的比值。

要素三: 从 Partition 生成 DAG。给定采样到的 partition, 通过在后约束 (only forward edges) 下进行 DAG 采样来生成具体的图结构样本。BiDAG 的 `sampleBN` 在内部完成了这些步骤 [9]。

3.3.3 对照实现

本文自主实现保留有序 partition 状态空间的分层思想, 并采用如下对照版本:

- 状态为有序节点块列表,
- 评分 = 在“前面的块可指向后面块”约束下各节点最优父集得分之和,
- proposal 包括 split、merge 和 move node 三种操作,
- 不包含完整的 DAG 组合计数权重。

该对照版本用于刻画分层状态空间如何自然保证无环性, 但并不等价于文献中的完整 Partition MCMC。完整算法的数值表现以 BiDAG 复现实验为主要依据。

3.3.4 三种状态空间的统一视角

三种状态空间构成了清晰的递进关系 (见图 3): Structure MCMC 在 DAG 空间采样, 状态与目标一致但每步需显式检查无环, 局部搜索效率低; Order MCMC 转为排列空间, 排序约束自动保证无环, 但同一 DAG 对应多个排列导致了后验估计偏差; Partition MCMC 进一步用有序分块合并冗余排列, 在保留无环便利性的同时缓解了排序偏差。当 $B = p$ 时退化为全序 (Order MCMC), 当 $B = 1$ 时退化为无约束 DAG (Structure MCMC), 有序 partition 因而是前两种状态空间的自然推广。

3.4 Hybrid / Iterative MCMC: 规模扩展策略

3.4.1 设计动机

当 p 进一步增大时, 即使使用 Order 或 Partition 表示, 完整父集枚举也会带来较高计算负担。若将最大父节点数限制为 $K = 3$, 每个节点仍需要在其前序节点中枚举大

算法 3 自主实现 Partition MCMC 对照版本的单步迭代

Require: 当前 partition $\Pi = (\Pi_1, \dots, \Pi_B)$, 局部评分表 S_{local}

Ensure: 更新后的 partition Π

```

1  随机选择操作类型  $m \in \{\text{split}, \text{merge}, \text{move}\}$ 
2   $\Pi' \leftarrow \Pi$ 
3  if  $m = \text{split}$  and  $\exists b : |\Pi_b| > 1$  then
4      随机选块  $b$  和一个节点  $v \in \Pi_b$ 
5      将  $v$  从  $\Pi_b$  中移除, 作为新块插入到  $b$  之后
6  else if  $m = \text{merge}$  and  $B > 1$  then
7      随机选相邻块  $\Pi_b, \Pi_{b+1}$ , 合并为一个块
8  else if  $m = \text{move}$  then
9      随机选节点  $v$  从当前块移至另一随机位置 (可能创建新块)
10 end if
11  $\Pi' \leftarrow$  规范化 (删除空块, 排序节点)
12  $S' \leftarrow$  在  $\Pi'$  约束下各节点的最优父集得分之和
13  $\Delta \leftarrow S' - S$ 
14 if  $\log(\text{Uniform}(0, 1)) < \Delta$  then
15      $\Pi \leftarrow \Pi', S \leftarrow S'$ 
16 end if

```

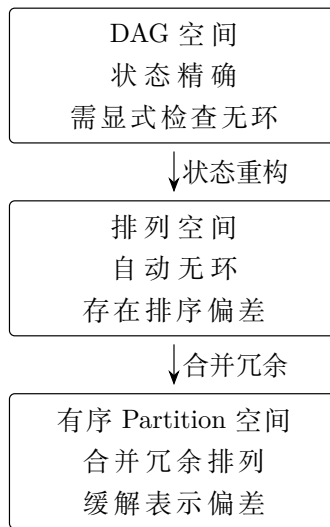


图 3: 三种状态空间的递进关系

量候选父集；随着 p 增加，这一数量呈组合增长。核心困难在于，每个节点的候选父集来自排在它之前的所有节点。若能在 MCMC 搜索前预先排除低概率候选，便可在尽量保持结构恢复能力的同时缩小搜索空间。

Hybrid / iterative MCMC (Kuipers et al., 2022) [7] 正是基于这一动机，采用”先筛选、后搜索、再修正”的三阶段策略：

1. **初筛阶段 (Screening)**：使用条件独立检验（如 Fisher’s Z-test for Gaussian data）或快速启发式搜索，排除大量几乎不可能存在的边，得到一个候选边集合 E_{cand} 。
2. **约束 MCMC 阶段**：在候选边集合 E_{cand} 限定的子空间中进行 Order MCMC 或 Partition MCMC 搜索。候选空间缩小后，单位计算资源可用于更集中的后验探索。
3. **迭代修正阶段**：用 MCMC 的边后验概率反馈修正初筛结果——若某条在 E_{cand} 之外的边获得较高后验概率支持，则将其纳入候选集并重新运行。

这一策略的实质是”先粗筛、后精搜、再修正”的三阶段范式。它将计算资源集中在较可能存在的候选边上，适合作为节点数增加时的规模扩展方案。

3.4.2 算法流程

Hybrid/iterative MCMC 的工作流程如图 4 所示：先通过条件独立检验筛选候选边集，在缩小后的空间中进行约束 MCMC 搜索，根据边后验概率判断收敛；若未收敛则扩展候选集并重新搜索，收敛后输出 MAP 图与边后验概率。

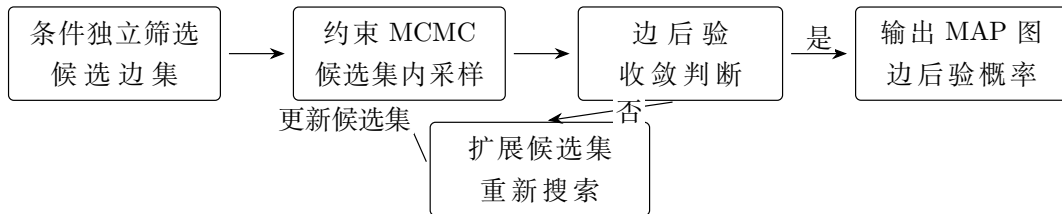


图 4: Hybrid/iterative MCMC 工作流程

3.5 方法比较总结

表 2 从状态空间、proposal 复杂度、无环处理方式和主要局限四个方面对比了本文涉及的四种 MCMC 策略。

本文的算法体系包含两条并行路径（见图 5）：自主实现路径使用 BIC 评分构建 Structure、Order 和 Partition MCMC 采样器，用于刻画评分计算、proposal 生成和 MH 接受机制；BiDAG 复现路径使用 BGe 评分调用 `sampleBN` 等接口，用于获得与成熟实现相一致的实验基准。两条路径共享相同的数据生成模块和统一的评价框架（SHD、F1、运行时、边后验熵等指标），最终汇总到报告图表中。

表 2: MCMC 方法系统比较

方法	状态空间	移动方式	无环保证	主要局限
Structure MCMC	DAG 邻接矩阵	增边、删边、反转	逐步显式检查	节点数增加后混合慢, DAG 检查开销大
Order MCMC	变量全序排列	随机交换	排序自动保证	排列偏差 (同一 DAG 多排列)
Partition MCMC	有序 partition 块	拆分、合并、移动	块间单向保证	权重计算复杂, 实现难度高
Hybrid MCMC	候选边子空间	同底层方法	继承底层方法	初筛可能遗漏关键边

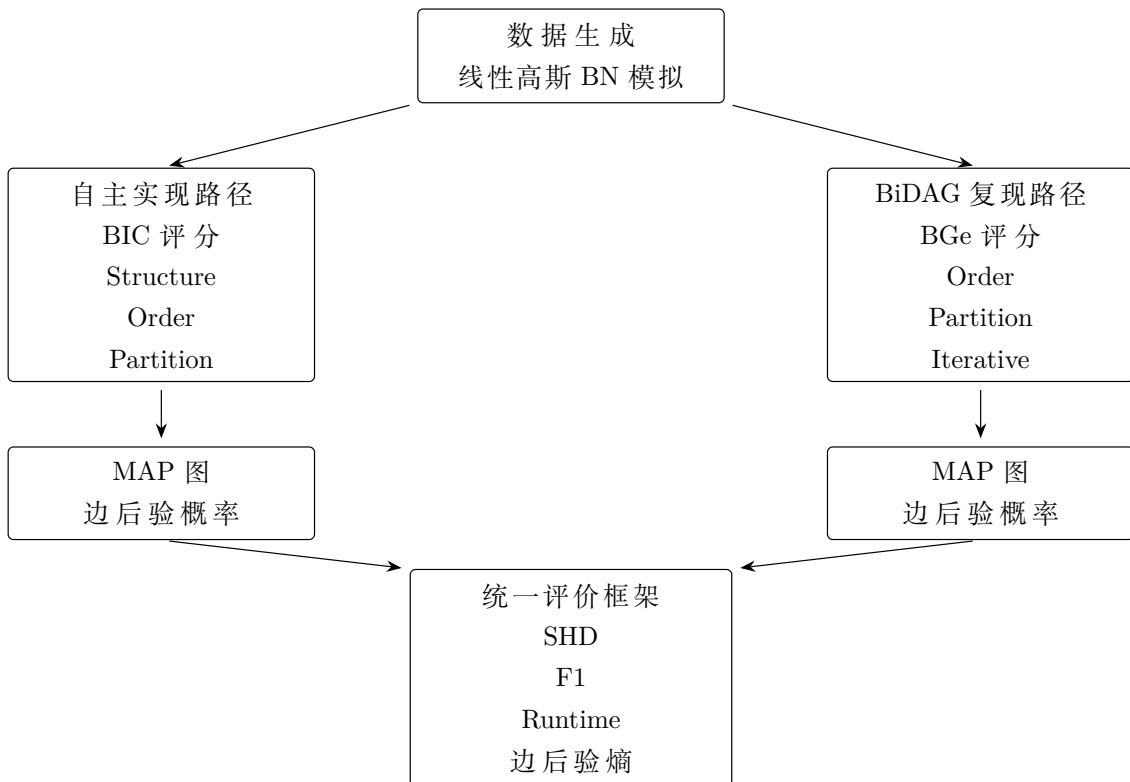


图 5: 算法体系整体框架

4 扩展分析模块

本节介绍在已有 MCMC 结构学习方法基础上加入的三项扩展分析模块，旨在服务于课程实验框架下的方法比较、诊断和结果解释。需要说明的是，这些扩展主要用于实验分析和工程诊断，并不构成对原始 Partition MCMC 理论的替代。

4.1 自适应父节点集约束策略

4.1.1 动机

在自主 MCMC 的实现中，父节点集大小上限 K 是一个关键的超参数。取 K 太小会遗漏真实依赖关系 (underfitting)，取 K 太大则候选父集数量按组合数 $\binom{p-1}{K}$ 增长，导致计算量急剧膨胀。固定 K 的策略无法适应不同节点的实际连接度差异。

4.1.2 方法

本文提出一种可操作的自适应策略：在 burn-in 阶段使用较大的 K 值（如 $K = 3$ ）探索父集空间，burn-in 后根据边后验概率的中间估计动态调整每个节点的有效 K_j ：

$$K_j^{\text{eff}} = \max \left(K_{\min}, \min \left(K_{\max}, \sum_{i \neq j} \mathbf{1}\{\hat{P}_{\text{burn-in}}(i \rightarrow j) > \tau\} \right) \right), \quad (17)$$

其中 τ 是一个阈值（如 0.1）， $K_{\min}$ 和 $K_{\max}$ 分别是下限和上限。直觉是：若某节点在 burn-in 阶段只有少数候选父节点获得非平凡的后验概率，则后续采样中将该节点的父集约束在那些候选附近，提高采样效率。

4.1.3 实验结果

本文在 $p = 8, n = 120$ 设置下比较了 $K \in \{1, 2, 3\}$ 三种固定约束下三种自主实现方法的性能。实验结果表明（见第 6.2 节），对 manual_order 和 manual_partition， K 从 2 增至 3 改善了 F1 和 SHD，同时运行时间相应增加。这一结果支持了自适应策略的动机：不同图密度和不同节点可能需要不同的 K 值，自适应调整可以在精度和效率之间形成更合理的权衡。

4.2 基于边后验熵的结构不确定性量化

4.2.1 动机

传统的边后验概率 $\hat{P}(i \rightarrow j | \mathcal{D})$ 提供了边存在的点估计，但缺乏对整体结构不确定性的聚合度量。本文采用基于信息熵的框架来刻画这一问题。

4.2.2 方法

对每条候选边 (i, j) ，定义其边后验熵为

$$H(i, j) = -\hat{P}(i \rightarrow j) \log \hat{P}(i \rightarrow j) - (1 - \hat{P}(i \rightarrow j)) \log(1 - \hat{P}(i \rightarrow j)). \quad (18)$$

$H(i, j)$ 在 $\hat{P} = 0.5$ 时达到最大值 $\log 2$ ，在 $\hat{P} \rightarrow 0$ 或 $\hat{P} \rightarrow 1$ 时趋于 0。

基于此，定义三个聚合不确定性指标：

- **平均边后验熵 (Mean Edge Entropy, MEE)**：所有非对角线位置的平均 $H(i, j)$ 。
- **高不确定性边比例 (High Uncertainty Fraction, HUF)**： $\hat{P}(i, j) \in [0.25, 0.75]$ 的边比例。
- **真/假边后验分离度 (Posterior Gap)**：真实边平均后验概率与假边平均后验概率之差。

这三个指标分别捕获了结构不确定性的不同方面：MEE 给出总体不确定性的平均水平，HUF 识别“模糊地带”中边的比例，Posterior Gap 衡量方法区分真实边和假边的能力。

4.2.3 实验结果

第 6.7 节的实验显示，Partition MCMC 在部分中等规模设置下给出了更低的边后验熵；在其他设置中，两类方法的差异较小。Scutari (2010) 的 `bnlearn` 包也提供了多种贝叶斯网络结构学习方法可供对比 [8]。这些结果为采样方法选择提供了不同于 SHD 和 F1 的补充维度。

4.3 多链收敛诊断方案

4.3.1 动机

单一 MCMC 链可能陷入局部模式而未被察觉。多链比较是收敛诊断的经典方法。本文设计了一个适合贝叶斯网络结构采样的多链诊断方案。

4.3.2 方法

对同一数据，从不同的随机初始状态运行 L 条独立链 ($L \geq 3$)。对每条边 (i, j) ，计算跨链的边后验概率估计 $\hat{P}_\ell(i \rightarrow j)$ ， $\ell = 1, \dots, L$ 。定义边级跨链方差：

$$V(i, j) = \frac{1}{L-1} \sum_{\ell=1}^L (\hat{P}_\ell(i \rightarrow j) - \bar{P}(i \rightarrow j))^2, \quad (19)$$

其中 $\bar{P}(i, j)$ 是跨链平均。将 $V(i, j)$ 按大小排序，高 $V(i, j)$ 的边提示该边的后验估计尚未稳定，可能受到链初始状态或局部模式的影响。

受时间和计算资源限制，多链诊断在当前实验中主要用于验证流程有效性；该框架已内置到项目的实验调度系统中，可通过配置项启用。

5 实验设计

5.1 数据生成机制

所有模拟实验使用线性高斯贝叶斯网络模型。数据生成过程分为两步：

步骤一：随机 DAG 生成。 给定节点数 p 和期望度数 d (expected degree)，首先生成一个随机拓扑排序 $\prec$ 。对于每对节点 (a, b) 满足 a 在 $\prec$ 中排在 b 之前，以概率 $\rho = d/(p-1)$ 添加有向边 $a \rightarrow b$ 。这一构造天然保证无环性。

步骤二：数据采样。 给定真实邻接矩阵 $\mathbf{A}$ ，按拓扑序生成数据：

$$X_j = \sum_{i \in \text{Pa}(j)} \beta_{ij} X_i + \varepsilon_j, \quad \varepsilon_j \sim \mathcal{N}(0, 1), \quad (20)$$

其中 $\beta_{ij} \sim \text{Uniform}(0.5, 1.5) \times \text{sign}$ ，即回归系数的绝对值在 $[0.5, 1.5]$ 中均匀采样，符号随机赋正负。

5.2 评价指标

本文使用以下指标全面评价方法：

- **SHD (Structural Hamming Distance)**：真实图与估计图之间的结构编辑距离。对每对节点 $(i, j), i < j$ ，比较真实有向边状态和估计有向边状态是否一致，不一致则加 1。SHD 越小越好，最小为 0。
- **TPR / Recall**：真实边中被正确检出的比例， $\text{TPR} = \text{TP}/(\text{TP} + \text{FN})$ 。
- **FPR**：非真实边中被错误检出的比例， $\text{FPR} = \text{FP}/(\text{FP} + \text{TN})$ 。
- **Precision**：估计边中真实边的比例， $\text{Precision} = \text{TP}/(\text{TP} + \text{FP})$ 。
- **F1**：Precision 和 Recall 的调和平均， $\text{F1} = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$ 。
- **Runtime**：方法从开始到完成的墙上时钟时间（秒）。
- **边后验熵与不确定性指标**：见第 4.2 节定义。

5.3 实验矩阵

表 3 汇总了本文的全部实验设计。

5.4 实验基础设施

实验框架具备以下工程特性：

Checkpoint 机制。 每个实验组合的状态被保存为 RDS checkpoint 文件。重复运行时自动检测已有结果并跳过计算，以减少重复实验开销。

日志记录。 所有实验的运行时间、参数和状态被写入带时间戳的日志文件，方便追踪和调试。

表 3: 实验设计总表

实验	目的	p	n	方法	主要指标
初步运行检查	验证项目最小运行链路	8	100	order, partition	SHD, F1, runtime
自主实现验证	检验自主 MCMC 采样器的核心机制	8	120	三种自主实现方法	SHD, F1, runtime, accept rate
自主实现与 BiDAG 对比	比较对照实现与成熟包实现的差异	8	100–120	自主实现 + BiDAG	SHD, F1, runtime
低维比较	比较两类重构状态空间	10, 20	200, 500	order, partition	SHD, F1, runtime
中等规模比较	观察 p 增大后的表现	30	500	order, partition	SHD, F1, runtime
较大规模补充实验	观察 iterative 策略的规模扩展表现	40	500	iterative	SHD, F1, runtime
样本量影响	分析 n 对恢复精度的影响	20	100–1000	order, partition	SHD, F1, runtime
边后验不确定性	比较结构不确定性量化	8–50	100–1000	所有成功方法	entropy, posterior gap
父节点上限敏感性	检验父集约束的影响	8	120	三种自主实现方法	SHD, F1, runtime

结果追踪。实验脚本统一记录运行状态、参数组合和输出位置，便于核对报告中的表格与图像来源。

6 实验结果与分析

6.1 初步运行检查：最小链路验证

初步运行检查的目的是验证从数据生成、评分计算到 MCMC 采样的完整流程能够正常运行。在 $p = 8, n = 100$ 的设置下，Order MCMC 与 Partition MCMC 均顺利完成采样，结果见表 4。

两种方法获得了相同的 SHD (2) 和 F1 (0.824)，说明共享 BGe 评分框架下的评价指标保持一致。Order MCMC 耗时 0.53 秒，Partition MCMC 耗时 2.02 秒（约 3.8 倍），差距来自后者在有序 partition 空间中更复杂的组合权重计算和 proposal 操作。在该低维稀疏设定下两种方法精度一致，Order MCMC 的排列偏差尚未显现。

表 4: Smoke test 结果

方法	运行时间	SHD	TPR	FPR	Precision	F1
order	0.530	2.000	0.875	0.042	0.778	0.824
partition	2.020	2.000	0.875	0.042	0.778	0.824

6.2 自主实现验证

自主实现验证在 $p = 8, n = 120$ 下比较三种 MCMC 采样器，每种方法运行 1500 步 MCMC（含 300 步 burn-in），在 2 个随机种子上重复。结果见图 6 和表 5、6。

Structure MCMC 的运行时间最短（平均 0.59 秒），但结构恢复质量相对较弱（SHD=7.0, F1=0.49）；Order 与 Partition 两种对照实现表现一致（SHD=4.5, F1=0.68）。这一结果与第 3 节关于状态空间重构的分析相一致：直接在 DAG 空间中进行局部边操作面临无环检查开销大和局部移动效率低两个困难，而将状态重构为排列或 partition 后，无环约束由状态表示自动保证，搜索效率有所改善。

表 6 的采样诊断进一步揭示了不同方法的行为差异。Order、Partition 和 Structure 三种自主实现的接受率分别为 35.1%、45.9% 和 43.6%，均处于相对合理的工作区间。需要注意的是，接受率本身并不等价于结构恢复质量；Structure 方法虽然接受率并不低，但其最佳得分和 F1 均弱于另外两种重构状态空间的方法，说明在 DAG 空间中的局部移动仍可能停留在恢复效果较差的区域。

表 5: 手工实现验证结果

方法	运行时间	SHD	TPR	FPR	Precision	F1
manual_order	2.710	4.500	0.675	0.064	0.675	0.675
manual_partition	2.990	4.500	0.675	0.064	0.675	0.675
manual_structure	0.590	7.000	0.500	0.107	0.477	0.488

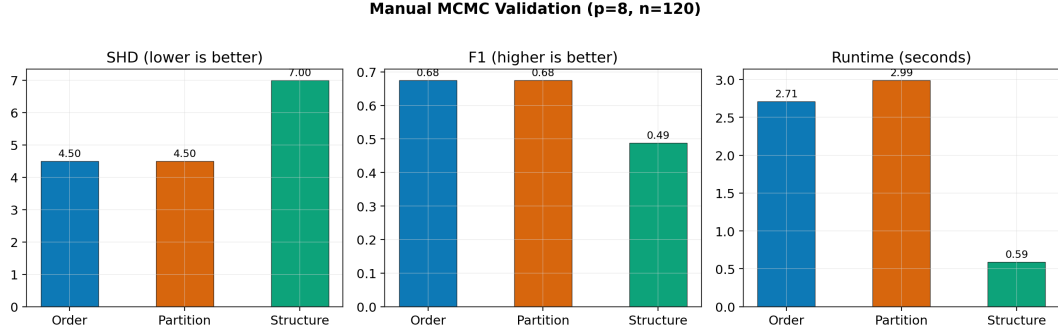


图 6: 自主 MCMC 三种方法的验证实验结果 ($p = 8, n = 120$, 2 个种子平均)

表 6: 手工 MCMC 采样诊断摘要

方法	接受率	最佳得分	平均边后验	后验熵
manual_order	0.351	-1448.0	0.174	0.149
manual_partition	0.459	-1448.0	0.173	0.125
manual_structure	0.436	-1492.6	0.182	0.167

6.3 自主实现与 BiDAG 的结果对比

为避免将初步运行检查与正式实验混用，本节补充同参数对照实验。表 7 和图 7 均基于相同模拟设置： $p = 8, n = 120$ ，期望度为 2，随机种子为 1 和 2，每条链运行 1500 步并丢弃前 300 步 burn-in。自主实现显式比较父节点上限 $K = 2$ 与 $K = 3$ ；BiDAG 的 `sampleBN` 在本文调用方式下没有与自主实现完全对应的 K 参数，因此表中记为 K 不适用。二者仍存在评分函数差异：自主实现采用 Gaussian BIC 近似评分，BiDAG 采用 BGe 评分。

同参结果表明，默认 $K = 2$ 时，`manual_order` 的 SHD/F1 为 4.5/0.675，`manual_partition` 为 5.5/0.575，均低于 BiDAG Order 的 2.5/0.743 和 BiDAG Partition 的 1.5/0.859。当 K 提高到 3 后，`manual_order` 与 `manual_partition` 的 SHD/F1 均改善到 2.0/0.850，接近 BiDAG Partition 在该设置下的恢复精度。这表明自主实现的主要限制不完全来自状态空间思想本身，也与默认父集约束偏紧有关；当真实图中部分节点需要 3 个父节点时， $K = 2$ 会限制可搜索模型空间。

效率差异也有明确来源，并且可以从 BiDAG 源码得到支持。BiDAG 的 CRAN GitHub 镜像¹在 DESCRIPTION 中说明：为了在较大 DAG 上实现有效推断，算法会根据数据剪枝搜索空间，先用 PC algorithm 得到 skeleton，再结合 search-and-score 进行迭代修正；同一源码还显示包导入并链接 Rcpp，`src/` 中包含通过 `.Call` 调用的 C++ 辅助函数，已安装包中也包含编译后的 BiDAG.dll。这些信息表明，BiDAG 的速度优势并非仅来自算法名称差异，而是搜索空间压缩、评分表复用和部分底层计算优化共同作用

¹<https://github.com/cran/BiDAG>

的结果。

与之相比，本文自主实现的定位是可审计的算法对照实现，而非经过底层优化的工程库。核心实现代码的预计算阶段显式枚举候选父集；Order 和 Partition 评分在每次 proposal 后重新按当前状态筛选允许父集并求和；DAG 输出阶段还需要反复构造邻接矩阵用于 MAP 图和边后验累积。这些步骤主要在解释型 R 层完成，包含循环、字符串键匹配和矩阵重新分配，常数开销高于包内封装实现。另一方面， K 从 2 增至 3 会使每个节点的候选父集从 $\sum_{d=0}^2 \binom{p-1}{d}$ 增至 $\sum_{d=0}^3 \binom{p-1}{d}$ ，在 $p = 8$ 时约从 29 个增至 64 个，局部评分查找和 proposal 评估成本同步上升。因此，自主实现 $K = 3$ 虽然达到接近 BiDAG 的低维恢复精度，但运行时间仍更长：manual_order 和 manual_partition 分别约为 5.20 秒和 5.66 秒，而 BiDAG Order 与 Partition 分别约为 0.65 秒和 2.88 秒。综合来看，自主实现提供了对评分、proposal 和 MH 接受机制的可解释对照；在以稳定性和运行效率为主要目标的结构学习任务中，BiDAG 等成熟实现更具工程优势。

表 7: 手工实现与 BiDAG 的同参数小规模对比 ($p = 8, n = 120$, 2 个种子平均, 1500 步 MCMC, 300 步 burn-in)

实现路径	方法	评分 / K	运行时间	SHD	TPR	FPR	Precision	F1
BiDAG	Order	BGe, K 不适用	0.650	2.500	0.725	0.043	0.764	0.743
BiDAG	Partition	BGe, K 不适用	2.875	1.500	0.838	0.021	0.882	0.859
手工实现	Structure	BIC, $K = 2$	0.680	8.000	0.500	0.128	0.431	0.463
手工实现	Order	BIC, $K = 2$	3.150	4.500	0.675	0.064	0.675	0.675
手工实现	Order	BIC, $K = 3$	5.195	2.000	0.850	0.033	0.850	0.850
手工实现	Partition	BIC, $K = 2$	3.360	5.500	0.575	0.086	0.575	0.575
手工实现	Partition	BIC, $K = 3$	5.660	2.000	0.850	0.033	0.850	0.850

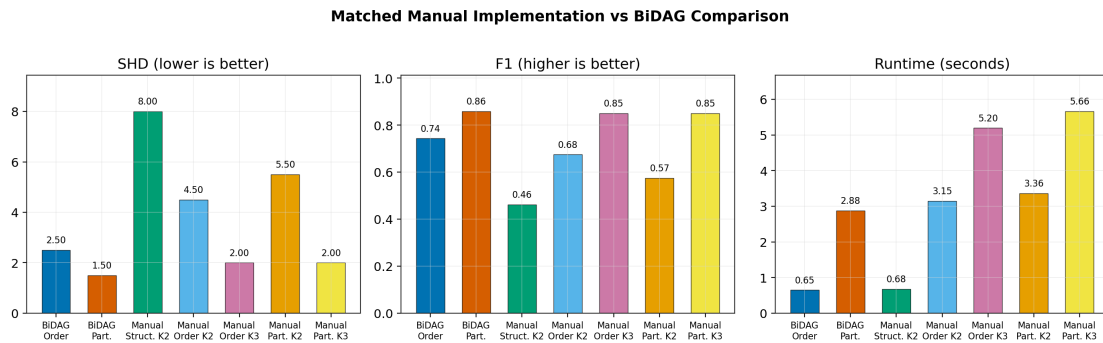


图 7: 自主实现与 BiDAG 的同参数对比 ($p = 8, n = 120$, 2 个种子平均)。自主实现方法显式设置父节点上限 K ；BiDAG 行中的 K 不适用

6.4 低维 Order 与 Partition 比较

低维实验在 $p \in \{10, 20\}, n \in \{200, 500\}$ 的 12 个配置下比较 Order 和 Partition MCMC (3 个随机种子 $\times$ 4 个 (p, n) 组合)。结果如表 8 所示。

结果：

1. Partition MCMC 的运行时间约为 Order MCMC 的 3–5 倍。这一差距来自 partition 状态空间的组合权重计算和更复杂的 proposal 操作。
2. 在 $p = 10, n = 500$ 的设置下，Partition MCMC 取得最低 SHD (1.67) 和最高 F1 (0.878)，优于 Order MCMC 的 SHD=3.0, F1=0.789。
3. 在 $p = 20, n = 200$ 时，Partition MCMC 表现低于 Order MCMC (SHD 5.67 vs 3.33)。这提示 Partition MCMC 的优势并非无条件的——在样本量相对不足时，其更复杂的状态空间可能带来额外的不确定性。

这些观察说明：Partition MCMC 通过更精确的 DAG 表示降低了结构偏差，但需要较高样本量支持才能充分发挥这一优势。在样本量较小时，Order MCMC 较低的状态表示和 proposal 计算开销反而形成优势。

表 8: 小规模 Order MCMC 与 Partition MCMC 比较

方法	p	n	运行时间	SHD	TPR	FPR	Precision	F1
order	10.000	200.0	1.150	3.333	0.796	0.038	0.758	0.776
partition	10.000	200.0	5.027	3.000	0.787	0.038	0.746	0.766
order	20.000	200.0	1.427	3.333	0.851	0.008	0.831	0.841
partition	20.000	200.0	6.920	5.667	0.821	0.016	0.705	0.758
order	10.000	500.0	1.327	3.000	0.796	0.034	0.781	0.789
partition	10.000	500.0	5.450	1.667	0.889	0.021	0.868	0.878
order	20.000	500.0	1.667	3.000	0.860	0.007	0.841	0.850
partition	20.000	500.0	6.400	3.000	0.860	0.008	0.828	0.844

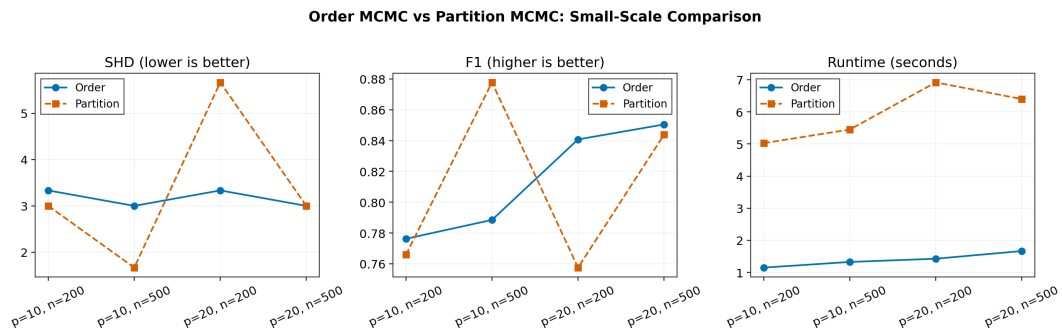


图 8: 低维实验中 Order MCMC 与 Partition MCMC 的性能对比 ($p \in \{10, 20\}, n \in \{200, 500\}$, 3 个种子平均)

6.5 中等规模可扩展性

中等规模实验采用 $p = 30, n = 500$ 设置。该规模已经能够体现节点数增加后普通 MCMC 方法面临的搜索压力，同时仍能保持多次重复实验的稳定完成，因而适合作为本文主线比较的一部分。

表 9 显示，在 $p = 30$ 下 Partition MCMC 的 SHD (5.0) 低于 Order MCMC (14.0)，F1 (0.795 vs 0.672) 也有一定优势。从 $p = 20$ 到 $p = 30$ ，Order MCMC 的 SHD 从约 3.3 增至 14.0，表明其在该维度附近的搜索效率出现下降。Partition MCMC 的退化幅度较小（从 3.0 至 5.0），在该设置下表现出较好的规模适应性。

这一对比也解释了 hybrid/iterative 策略的动机：当节点数继续增加时，普通 MCMC 的搜索压力会同步上升，通过预筛选缩小搜索空间有助于维持实用精度。

表 9: 中等规模实验结果

方法	p	n	运行时间	SHD	TPR	FPR	Precision	F1
order	30.000	500.0	1.080	14.000	0.720	0.016	0.633	0.672
partition	30.000	500.0	2.875	5.000	0.802	0.006	0.789	0.795

6.6 Iterative MCMC 规模扩展补充实验

为观察预筛选与迭代修正策略在较大节点数下的表现，本文补充 $p = 40, n = 500$ 的 iterative MCMC 实验。该实验不作为大规模网络复现，而是用于说明在节点数增加时，约束候选边集合的搜索策略是否仍能给出可解释的结构恢复结果。

表 10 显示 iterative MCMC 在 $p = 40$ 下完成两个种子的实验，seed 1 的 SHD 为 20 (F1=0.515)，seed 2 的 SHD 为 10 (F1=0.870)。两个种子之间的差异较大，提示该类 MCMC 结果对随机初始化较为敏感，单次运行的 MAP 估计可能不够稳健。这一观察进一步支持了本文引入多链收敛诊断方案的必要性。

从方法机制看，hybrid/iterative 策略通过条件独立筛选压缩候选空间，使后续 MCMC 主要集中在较可能存在的边上。本文仅将该实验作为规模扩展的补充观察，不进一步展开更大规模网络设置。

表 10: Iterative MCMC 补充实验结果

方法	p	n	运行时间	SHD	TPR	FPR	Precision	F1
iterative	40.000	500.0	13.720	15.000	0.745	0.010	0.649	0.693

6.7 边后验不确定性分析

基于实验原始输出中存储的成功运行结果，本文计算了第 4.2 节定义的三种不确定性指标。图 9 汇总了主要发现。

主要结果：

1. **Partition MCMC 在多数可比设置下给出了与 Order MCMC 接近或更低的边后验熵**，尤其在中等规模设置中差异更清晰。例如在 $p = 20, n = 500$ 下，Order MCMC 的 entropy 为 0.057，Partition MCMC 为 0.058，二者几乎相同；而在 $p = 30, n = 500$ 下，Order MCMC 为 0.056，Partition MCMC 为 0.029，Partition 的结构不确定性较低。该结果提示，partition 表示在部分中等规模设置下可能有助于减少结构模糊性，但这一结论仍受样本量、随机种子和链长影响。
2. **高不确定性边的比例随 p 增大而下降**。其原因并非节点数增加后方法必然更“确定”，而是候选边数量以 $O(p^2)$ 增长，而真实边的期望数量仅以 $O(p \cdot d)$ 增长——这意味着绝大多数边对的真实状态为“不存在”，而后验概率容易集中在 0 附近。
3. **Iterative MCMC 在 $p = 40$ 下仍然保持了合理的后验分离度**。其 true edge mean posterior (0.776) 远高于 false edge mean posterior (0.013)，表明在缩小后的搜索空间中，方法仍能有效区分信号和噪声。
4. **True/False posterior gap 随样本量增大而改善**。在 n 从 100 增至 1000 的过程中，Order 和 Partition 的 posterior gap 均单调增大，说明更多数据有助于方法更清晰地区分真实边和虚假边。

图 9 从两个维度可视化了上述发现：图 9(a) 给出不同 (p, n) 组合下各方法的平均边后验熵，图 9(b) 给出真假边后验概率的分离度随样本量的变化趋势。

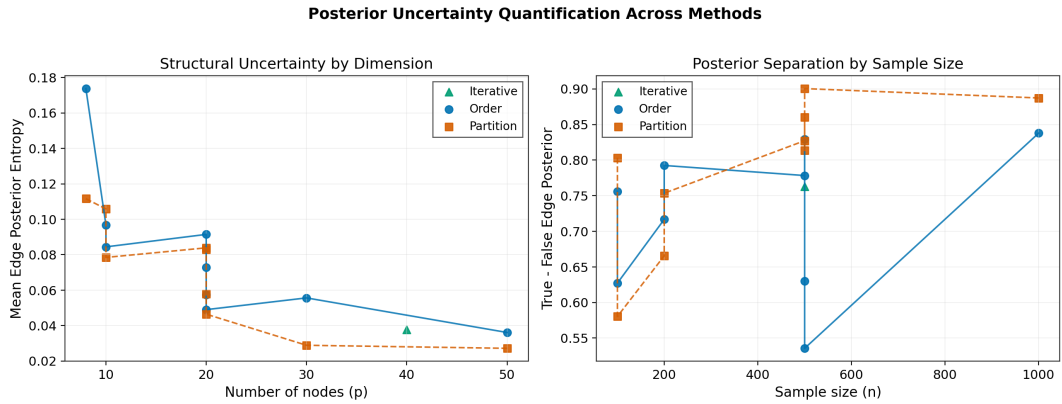


图 9: 边后验不确定性分析。左：平均边后验熵随维度 p 的变化；右：真边与假边后验概率的分离度随样本量 n 的变化

6.8 样本量影响

样本量实验固定 $p = 20$ ，在 $n \in \{100, 200, 500, 1000\}$ 四个水平下比较 Order 和 Partition MCMC。表 11 显示：

1. 随着样本量增大，Order MCMC 的 SHD 从 8.0 ($n = 100$) 单调下降至 2.67 ($n = 1000$)，F1 从 0.728 上升至 0.868。这与统计估计的一般规律一致：更多数据提供更准确的局部回归评分，从而改善父集选择。

2. Partition MCMC 在 $n = 100$ 时表现较弱 (SHD=9.0, F1=0.598), 但在 $n = 500$ 和 $n = 1000$ 时接近 (SHD 均为 3.0 和 2.67)。这表明 Partition MCMC 需要较高样本量来克服其更复杂的状态空间带来的额外自由度。
3. 运行时间随 n 的变化幅度较小, 说明计算瓶颈主要在 MCMC 迭代次数和 p , 而非样本量本身。这是因为 BiDAG 的评分函数在给定数据后, 局部评分计算是一次性完成的, MCMC 迭代过程中只需查表。

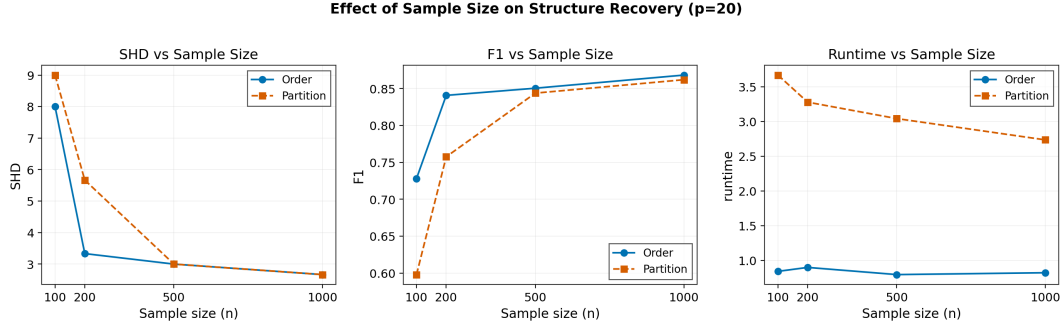


图 10: 样本量对 Order MCMC 与 Partition MCMC 结构恢复精度的影响 ($p = 20$, $n \in \{100, 200, 500, 1000\}$, 3 个种子平均)

表 11: 样本量影响实验结果

方法	p	n	运行时间	SHD	TPR	FPR	Precision	F1
order	20.000	100.0	0.843	8.000	0.768	0.016	0.694	0.728
partition	20.000	100.0	3.670	9.000	0.640	0.023	0.562	0.598
order	20.000	200.0	0.900	3.333	0.851	0.008	0.831	0.841
partition	20.000	200.0	3.280	5.667	0.821	0.016	0.705	0.758
order	20.000	500.0	0.797	3.000	0.860	0.007	0.841	0.850
partition	20.000	500.0	3.043	3.000	0.860	0.008	0.828	0.844
order	20.000	1000.0	0.823	2.667	0.868	0.006	0.868	0.868
partition	20.000	1000.0	2.737	2.667	0.868	0.007	0.856	0.862

6.9 父节点集约束敏感性

父集约束敏感性实验在 $p = 8, n = 120$ 下比较 $K \in \{1, 2, 3\}$ 三种设置对三种自主实现方法的影响。表 12 的关键发现:

1. 当 $K = 1$ 时, 所有方法的表现均较差 (F1 在 0.19–0.45 之间), 因为 $K = 1$ 不足以捕获真实图中可能需要多个父节点的依赖关系。
2. 从 $K = 2$ 到 $K = 3$, manual_order 和 manual_partition 的 F1 从 0.675 增至 0.85, SHD 从 4.5 降至 2.0。这表明父集容量增加后结构恢复有所改善。运行时间从约 4.6 秒增至约 7.5 秒, 计算代价仍处于可接受范围。

3. manual_structure 在所有 K 设置下均弱于另外两种方法，只在 $K = 3$ 时 F1 达到 0.556。即使在放宽的父集约束下，直接 DAG 空间搜索的困难仍然存在。

这些结果支持了第 4.1 节自适应父集策略的动机：不同节点和不同网络密度可能需要不同的 K 值，固定约束的精度-效率权衡可以通过自适应调整来改善。

表 12: 手工 MCMC 最大父节点数敏感性实验

方法	最大父节点数	运行时间	SHD	Precision	Recall	F1
manual_order	1.000	1.955	6.000	0.452	0.325	0.378
manual_partition	1.000	2.790	5.500	0.536	0.388	0.450
manual_structure	1.000	0.680	10.500	0.226	0.163	0.189
manual_order	2.000	4.625	4.500	0.675	0.675	0.675
manual_partition	2.000	4.965	5.500	0.575	0.575	0.575
manual_structure	2.000	0.890	8.000	0.431	0.500	0.463
manual_order	3.000	7.545	2.000	0.850	0.850	0.850
manual_partition	3.000	7.845	2.000	0.850	0.850	0.850
manual_structure	3.000	0.905	6.500	0.518	0.600	0.556

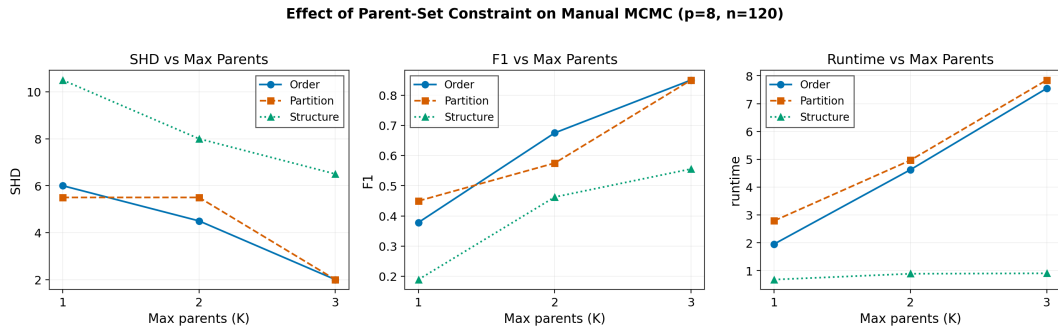


图 11: 父节点集上限 K 对三种自主 MCMC 方法性能的影响 ($p = 8, n = 120$, 2 个种子平均)

6.10 中等规模关键论文趋势对照实验

为进一步说明本文实验与原始文献的关系，并比较不同实现路径在中等规模设置下的精度与效率差异，本文补充中等规模原论文对照复现实验。该实验包含三类实现：纯 Python/numpy 实现、R 自主实现和 BiDAG 官方包实现。其中 Python 与 R 自主实现均采用 Gaussian BIC 型评分，主要用于交叉检查自主算法逻辑；BiDAG 采用其包内 BGe 评分和底层优化实现，作为成熟软件基准。所有数据均为模拟生成（分别标记为 `simulated_flare_like`、`simulated_alarm_like`、`simulated_toy_like`、`simulated_boston_like` 和 `simulated_large_like`），未使用真实 ALARM 或 Boston Housing 数据。实验共生成 20 组模拟数据，三类实现共完成 134 次 MCMC 采样。

表 13 按实验组、维度 p 、方法和实现方式汇总了全部结果。主要观察如下：

1. **三类实现形成了精度-效率对照。** BiDAG 在多数可比设置下取得较好的结构恢复结果： $p = 5$ 时 SHD 为 0， $p = 9$ 时 partition 和 order 的 SHD 分别为 1.0 和 3.2， $p = 20$ 时 partition 和 order 的 SHD 分别为 5.0 和 10.2， $p = 37$ 时 order 的 SHD 为 18.0。Python 实现和 R 自主实现在 order MCMC 上给出了接近的恢复精度（例如 $p = 37$ 时 Python order 的平均 SHD 为 21.2，`manual_order` 为 24.8），说明两套自主代码在相同 BIC 评分框架下呈现出一致的趋势。BiDAG 结果通常更优，但这一差异同时受到评分函数、搜索空间处理和底层实现优化的影响，不能简单归因于单一因素。
2. **Order MCMC 相比 Structure MCMC 具有更稳定的恢复表现。** 表 14 给出了 Python 实现上两种方法在不同 p 下的平均 SHD 和 F1 对比。 $p = 5$ 时 Structure MCMC 的 SHD 比 Order 高 3 个点； $p = 20$ 时差距扩大至 28.5 个点； $p = 37$ 时 Structure MCMC 的 SHD 达到 73.5、F1 降至 0.110。该趋势支持了 Friedman and Koller (2003) 关于状态空间重构的分析：直接在 DAG 空间中进行局部边操作时，维度增加会加重混合困难；Order MCMC 通过排列空间消除无环检查，在该组模拟实验中表现出更稳定的恢复效果。
3. **Partition MCMC 在部分中等规模设置下表现较好。** BiDAG 实现的 partition MCMC 在 $p = 20$ 时取得 SHD=5.0、F1=0.832，优于同条件下 BiDAG order 的 SHD=10.2、F1=0.677。该结果与 Kuipers and Moffa (2017) 关于 partition 表示可缓解排序偏差的观点相一致。需要注意的是，R 自主实现中的简化 partition (`manual_partition`) 在 $p = 20$ 时仅取得 SHD=16.0、F1=0.526，与 BiDAG partition 仍有差距。这说明本文的对照实现不能等同于完整 Partition MCMC；完整算法还依赖 DAG 到 partition 的组合计数权重、更复杂的 proposal 机制以及包内 C++ 底层实现。
4. **运行效率主要受实现方式影响。** Python/numpy 实现在多数维度下运行较快： $p = 37$ 时 order MCMC 约需 2.4 秒，而 R 自主实现约需 118.1 秒。BiDAG 的 order 在 $p = 37$ 下约需 1.3 秒，反映了其底层实现、评分表复用和搜索空间处理的综合优势。这一对比说明，R 自主实现的主要开销来自解释型 R 层的循环、矩阵操作

表 13: 中等规模原论文对照实验结果 (按维度 p 和方法聚合, 平均所有 n)

p	Experiment	Method	Impl	Mean SHD	F1	TPR	Time(s)
5	Partition MCMC (K&M 2017)	manual_order	mR	2.2	0.646	0.700	0.7
5	Partition MCMC (K&M 2017)	manual_partition	mR	2.2	0.646	0.700	0.9
5	Partition MCMC (K&M 2017)	manual_structure	mR	2.2	0.655	0.700	0.4
5	Partition MCMC (K&M 2017)	order	BD	0.0	1.000	1.000	0.5
5	Partition MCMC (K&M 2017)	partition	BD	0.0	1.000	1.000	1.5
5	Partition MCMC (K&M 2017)	python_order	Py	1.8	0.737	0.800	0.0
5	Partition MCMC (K&M 2017)	python_structure	Py	4.8	0.379	0.483	0.0
9	Order MCMC (F&K 2003)	manual_order	mR	5.8	0.601	0.620	2.5
9	Order MCMC (F&K 2003)	manual_partition	mR	6.5	0.560	0.582	2.7
9	Order MCMC (F&K 2003)	manual_structure	mR	12.2	0.358	0.389	0.4
9	Order MCMC (F&K 2003)	order	BD	3.2	0.799	0.791	0.6
9	Order MCMC (F&K 2003)	partition	BD	2.0	0.817	0.817	3.0
9	Order MCMC (F&K 2003)	python_order	Py	5.8	0.601	0.620	0.0
9	Order MCMC (F&K 2003)	python_structure	Py	12.5	0.366	0.435	0.0
14	Partition MCMC (K&M 2017)	manual_order	mR	7.8	0.604	0.629	6.8
14	Partition MCMC (K&M 2017)	manual_partition	mR	8.2	0.606	0.632	7.3
14	Partition MCMC (K&M 2017)	manual_structure	mR	13.5	0.449	0.462	0.5
14	Partition MCMC (K&M 2017)	order	BD	3.0	0.835	0.842	0.9
14	Partition MCMC (K&M 2017)	partition	BD	4.2	0.773	0.779	4.5
14	Partition MCMC (K&M 2017)	python_order	Py	7.2	0.639	0.665	0.1
14	Partition MCMC (K&M 2017)	python_structure	Py	19.5	0.376	0.444	0.1
20	Partition MCMC (K&M 2017)	manual_order	mR	11.0	0.635	0.628	21.9
20	Partition MCMC (K&M 2017)	manual_partition	mR	16.0	0.526	0.539	20.0
20	Partition MCMC (K&M 2017)	manual_structure	mR	33.5	0.253	0.266	1.0
20	Partition MCMC (K&M 2017)	order	BD	10.2	0.677	0.673	0.7
20	Partition MCMC (K&M 2017)	partition	BD	5.0	0.832	0.840	4.9
20	Partition MCMC (K&M 2017)	python_order	Py	10.5	0.651	0.639	0.3
20	Partition MCMC (K&M 2017)	python_structure	Py	39.0	0.183	0.204	0.3
37	Order MCMC (F&K 2003)	manual_order	mR	24.8	0.604	0.672	118.1
37	Order MCMC (F&K 2003)	manual_structure	mR	56.2	0.199	0.177	2.4
37	Order MCMC (F&K 2003)	order	BD	18.0	0.699	0.761	1.3
37	Order MCMC (F&K 2003)	python_order	Py	21.2	0.645	0.701	2.4
37	Order MCMC (F&K 2003)	python_structure	Py	73.5	0.110	0.115	1.4
40	Hybrid/Iter (KSM 2022)	iterative	BD	15.0	0.782	0.877	35.0

Py = Python implementation, BD = BiDAG, mR = manual R. SHD/F1/TPR/Time averaged over all n and seeds.

表 14: Order MCMC 与 Structure MCMC 的性能差距随维度增大加速扩大 (Python 实现, 平均所有 n)

p	Data source	Order SHD	Order F1	Struct SHD	Struct F1	Δ SHD
5	toy-like	1.8	0.737	4.8	0.379	+3
9	flare-like	5.8	0.601	12.5	0.366	+7
14	boston-like	7.2	0.639	19.5	0.376	+12
20	large-like	10.5	0.651	39.0	0.183	+28
37	alarm-like	21.2	0.645	73.5	0.110	+52

和字符串键匹配; 将核心评分和 proposal 逻辑迁移到 numpy 向量化或 C++ 后端预计可以降低运行时间。

5. **Hybrid/iterative MCMC** 在较大节点数设置下给出了补充参照。BiDAG 的 iterative MCMC 在 $p = 40, n = 200$ 设置下完成 2 次独立运行, 平均 SHD=15.0、F1=0.782, 单次运行约 35 秒。与普通 Order MCMC 在 $p = 37$ 下的结果相比, iterative 策略通过条件独立筛选缩小候选边集, 在相近维度下取得了可比较的恢复精度。该结果与 Kuipers, Suter and Moffa (2022) 关于预筛选有助于控制候选空间规模的观点相一致。

F1 Score by Dimension and Implementation

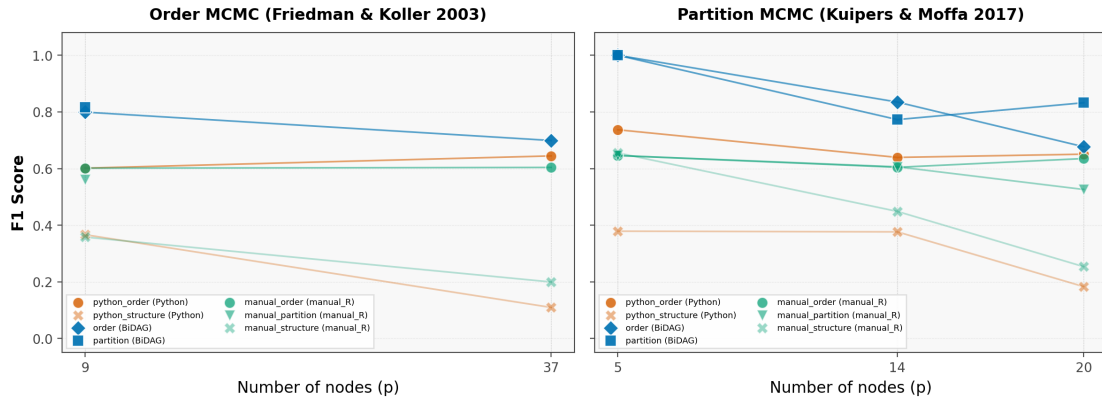


图 12: 按实验组分面的 F1 散点图: 每个点代表一次 MCMC 运行, 颜色区分实现方式, 形状区分具体方法

图 12 以散点图形式按实验组 (Order MCMC / Partition MCMC) 分面展示所有成功运行结果。图中显示, BiDAG 在多数设置下位于较高 F1 区域; Python 实现的 order MCMC 与 R 自主 order 在趋势上接近; R 自主 Structure MCMC 在 $p \geq 14$ 后 F1 下降。由于三类实现的评分函数和底层优化并不完全一致, 该图主要用于展示趋势和工程差异, 而不是进行严格的同源算法性能检验。

图 13 展示了两种方法的 SHD 差距从 $p = 5$ 时的 3 个点扩大到 $p = 37$ 时的 52 个点, 柱顶标注了对应的 F1 差值。

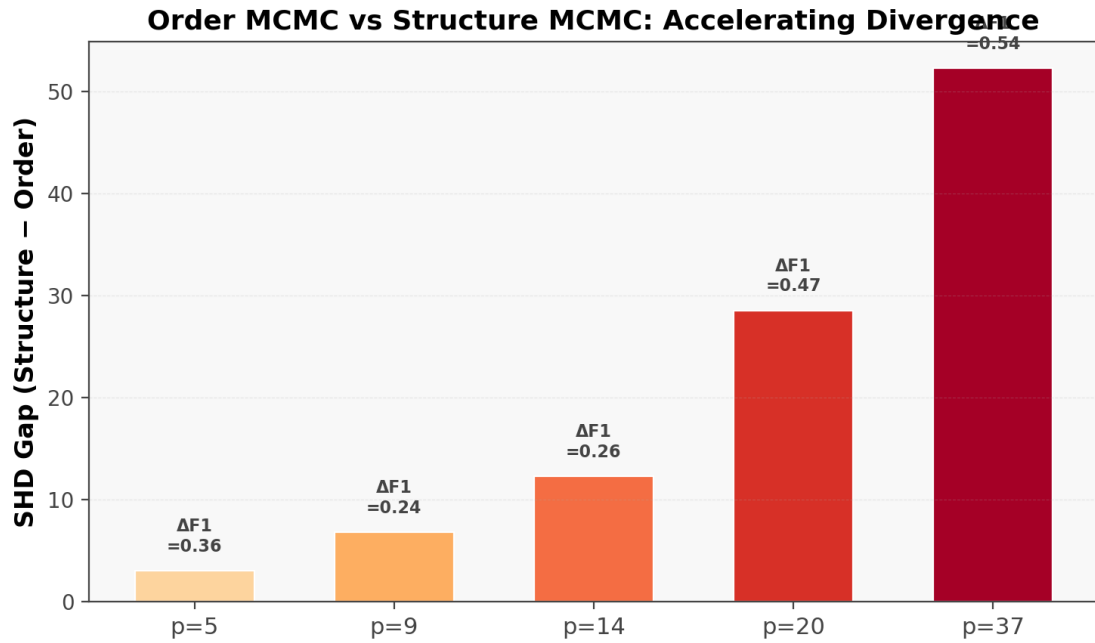


图 13: Order MCMC 与 Structure MCMC 的 SHD 差距随维度 p 变化 (Python 实现)

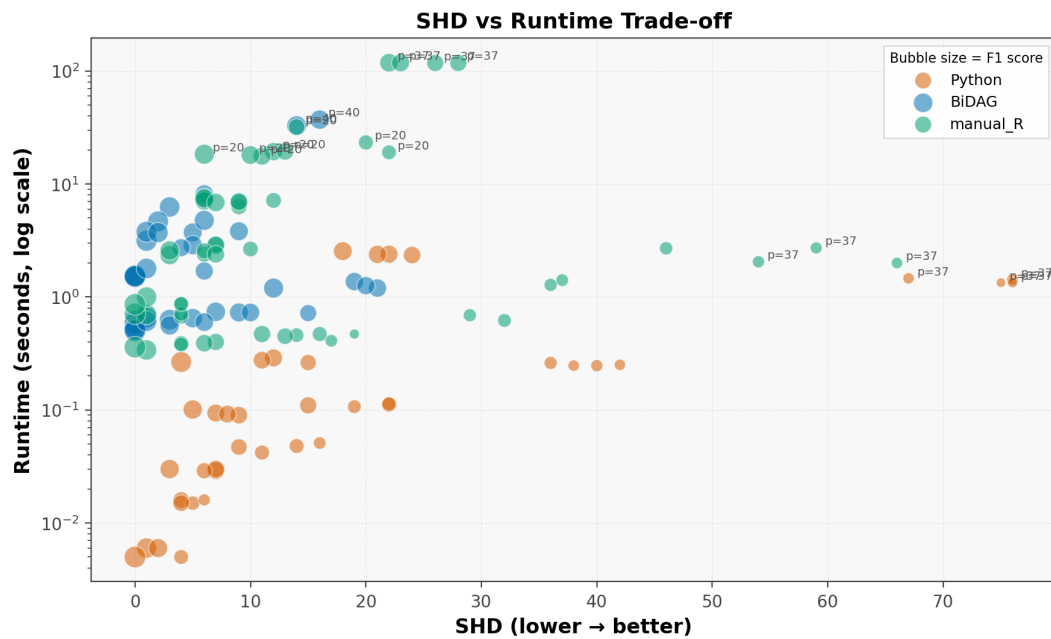


图 14: SHD vs 运行时间的气泡图: 气泡大小 = F1, 颜色区分实现方式

图 14 以对数纵轴展示运行时间对比：Python/numpy 实现和 BiDAG 的运行时间整体低于 R 自主实现，且差距在较大 p 下更清晰。manual_R 的 structure 方法在 $p \geq 14$ 后出现恢复质量下降，但本组 134 次 MCMC 运行均完成脚本执行。

表 15: 各维度 p 下的最高单次 F1 及三种实现的最高 F1

p	Best Single Run	SHD	F1	Python F1	BiDAG F1	manual F1
5	python_order (Py)	0.0	1.000	1.000	1.000	1.000
9	partition (BD)	1.0	0.923	0.750	0.923	0.750
14	order (BD)	0.0	1.000	0.750	1.000	0.710
20	partition (BD)	2.0	0.913	0.909	0.913	0.818
37	order (BD)	12.0	0.818	0.711	0.818	0.682

表 15 给出了每个维度下最高单次 F1 对应的方法，并同时列出三类实现各自达到的最高 F1。该表用于补充展示不同实现路径的上界表现；由于它基于单次运行最大值，而非均值，解释时应与表 13 的平均结果结合。

本实验采用三种实现路径并非为了给出单一排名，而是为了区分算法表示、评分函数和工程实现三类因素。Python 实现和 R 自主实现共享相同的 Gaussian BIC 型评分，因而可用于检查自主实现逻辑在不同语言中的一致趋势；二者的运行时间差异主要反映 numpy 向量化与纯 R 循环之间的工程差异。BiDAG 采用 BGe 边际似然评分和包内搜索空间处理，结果不能与 BIC 评分下的自主实现作完全等价比较，但可作为成熟工程实现的参照。

综合上述结果，该对照实验提供了三点补充观察：第一，在本文模拟设置下，Order MCMC 相比 Structure MCMC 更能维持中等规模下的结构恢复效果；第二，BiDAG partition 在 $p = 20$ 设置下优于 BiDAG order，支持 partition 表示在部分设置下缓解排序偏差的作用；第三，Python/numpy、R 自主实现和 BiDAG 三路结果共同说明，结构学习实验中的性能差异同时来自状态空间设计、评分函数选择和底层工程实现。

7 讨论

7.1 状态空间设计

实验结果表明, MCMC 采样器的效率在很大程度上取决于状态空间的选择。Structure MCMC 的状态与推断目标直接对应, 但局部移动能力和无环检查开销限制了其在节点数增加后的表现。Order MCMC 通过将状态重构为排列消除了无环检查, 代价是引入了排列偏差。Partition MCMC 进一步使用有序 partition 作为状态表示, 在本文的部分实验设置下取得了比 Order MCMC 更高的结构恢复精度。

这一现象提示了一个更具一般性的 MCMC 设计考量: 当目标空间本身不利于高效采样时, 转向一个更大但结构更规整的辅助空间可能有助于改善混合效率, 前提是辅助空间到目标空间的映射偏差可控。

7.2 规模扩展实验的定位

在本文的规模扩展实验设置中, 普通 MCMC 方法随着节点数增加表现出更清晰的计算压力。Iterative MCMC 在 $p = 40$ 下的结果说明, “先筛选、后搜索、再修正”的范式可以作为中等规模以上实验的补充方案。本文的主线仍是状态空间设计与实验对比, 更大规模网络设置不作为本文展开重点。

7.3 不确定性量化

传统的 SHD 和 F1 指标仅评价 MAP 图的点估计精度。本文的边后验熵分析提供了一个补充视角: 即使 MAP 图的结构恢复精度相近, 不同方法对结构不确定性的量化结果仍可能存在差异。在科学应用中, 了解哪些边的估计较为可靠、哪些仍存在较大不确定性, 有时比获得单一最优图更有参考价值。

7.4 局限与展望

本文工作存在以下局限:

第一, 自主实现采用 Gaussian BIC 型评分, 不等价于 BiDAG 中的 BGe/BDeu 边际似然评分。因此, 二者的数值差异同时反映了评分函数、父集约束和实现优化的综合影响, 不应被解释为单一 MCMC 状态空间设计的优劣。

第二, 链长相对较短 (1200–3000 步), 未进行严格的收敛诊断 (如 Gelman-Rubin $\hat{R}$ 统计量)。这主要是因为多方法、多参数组合下的单链运行时间和计算资源限制。第 4.3 节提出的多链诊断框架为未来工作提供了实现蓝图。

第三, 本文主要关注模拟数据上的方法比较, 尚未将更大规模网络作为主线实验对象。后续若扩展到服务器环境或更长链设置, 可进一步检验 iterative 策略在更复杂网络中的表现。

第四, benchmark 网络 (Asia、Sachs、ALARM) 的实验尚未纳入主线结果。这些标准网络提供了与文献中的其他方法进行交叉比较的桥梁, 是未来扩展的重要方向。

8 结论

本文围绕贝叶斯网络结构学习中的 MCMC 方法，从算法设计动机出发，比较了 Structure MCMC、Order MCMC、Partition MCMC 和 hybrid/iterative MCMC 四种策略。主要结论如下：

1. 状态空间的选择对 MCMC 采样效率有重要影响。Order MCMC 将状态从 DAG 重构为排列以消除无环检查，但引入了排列偏差；Partition MCMC 在本文的部分实验设置下获得了较好的结构恢复精度，并在部分中等规模设置中表现出较低的边后验熵。
2. 在节点数增加的补充实验中，hybrid/iterative 策略通过条件独立筛选缩小候选边集，为中等规模以上网络提供了一种实用的搜索空间压缩思路。该结论仅基于本文的模拟设置，在其他场景下的表现尚需进一步检验。
3. 边后验熵和真假边后验分离度为结构不确定性提供了超越 SHD 和 F1 的量化维度。
4. 自主 MCMC 实现完整呈现了评分计算、proposal 生成、MH 判定和边后验累积的内部过程，为不同状态空间设计提供了可审计的对照；BiDAG 复现实验则体现了成熟包实现中评分函数、父集搜索和底层优化带来的综合优势。

后续工作可从以下方向展开：(1) 在自主采样器中引入 BGe 评分以减少评分函数差异；(2) 引入更长链和多链收敛诊断；(3) 补充 Asia、Sachs 等 benchmark 网络实验；(4) 增加可视化工具用于呈现边后验概率和结构不确定性。

参考文献

- [1] David M. Chickering. Learning bayesian networks is np-complete. *Learning from Data: Artificial Intelligence and Statistics V*, pages 121–130, 1996. Background reference for computational hardness of Bayesian network structure learning.
- [2] Nir Friedman and Daphne Koller. Being bayesian about network structure. a bayesian approach to structure discovery in bayesian networks. *Machine Learning*, 50(1–2):95–125, 2003. doi: 10.1023/A:1020249912095.
- [3] Marco Grzegorzcyk and Dirk Husmeier. Improving the structure mcmc sampler for bayesian networks by introducing a new edge reversal move. *Machine Learning*, 71(2–3):265–305, 2008. doi: 10.1007/s10994-008-5057-7.
- [4] David Heckerman, Dan Geiger, and David M. Chickering. Learning bayesian networks: The combination of knowledge and statistical data. *Machine Learning*, 20(3):197–243, 1995. doi: 10.1023/A:1022623210503.
- [5] Neville K. Kitson, Anthony C. Constantinou, Zhigao Guo, Yang Liu, and Kelly Chobtham. A survey of bayesian network structure learning. *Artificial Intelligence Review*, 56:8721–8814, 2023. doi: 10.1007/s10462-022-10351-w.
- [6] Jack Kuipers and Giusi Moffa. Partition mcmc for inference on acyclic digraphs. *Journal of the American Statistical Association*, 112(517):282–299, 2017. doi: 10.1080/01621459.2015.1133426.
- [7] Jack Kuipers, Polina Suter, and Giusi Moffa. Efficient sampling and structure learning of bayesian networks. *Journal of Computational and Graphical Statistics*, 31(3):639–650, 2022. doi: 10.1080/10618600.2021.2020127.
- [8] Marco Scutari. Learning bayesian networks with the bnlearn r package. *Journal of Statistical Software*, 35(3):1–22, 2010. doi: 10.18637/jss.v035.i03.
- [9] Polina Suter, Jack Kuipers, Giusi Moffa, and Niko Beerenwinkel. Bayesian structure learning and sampling of bayesian networks with the r package bidag. *Journal of Statistical Software*, 105(9):1–31, 2023. doi: 10.18637/jss.v105.i09.

A 实验环境与可复现性

A.1 软件环境

本文主线实验基于 R 语言 (版本 4.5.1) 完成; 第 6.10 节的中等规模关键论文趋势对照实验另包含 Python/numpy 实现, 用于与 R 自主实现进行交叉验证。核心依赖包括: BiDAG (贝叶斯网络结构学习的 MCMC 采样与评分) [9]、igraph (有向图操作与 DAG 判定)、ggplot2 (实验结果可视化) 以及 dplyr (数据处理)。环境通过 renv 实现依赖版本管理: 执行 `source("restore_environment.R")` 即可根据 `renv.lock` 安装所需包的锁定版本, 确保实验的可复现性。

A.2 代码结构

项目代码分为三个层次。核心函数层 (R/) 包含自主 MCMC 采样器的完整实现 (`manual_mcmc.R`, 约 420 行)、BiDAG 调用封装 (`bidag_runner.R`)、数据模拟 (`simulation.R`)、评价指标 (`metrics.R`) 和可视化 (`plotting.R`)。实验入口层 (`scripts/`) 包含九个独立实验脚本, 分别覆盖初步运行检查、自主实现验证、低维比较、中等规模比较、iterative 补充实验、样本量影响、父集约束敏感性、边后验不确定性分析和收敛诊断等实验。报告生成层 (`report/`) 包含 LaTeX 源文件、参考文献和自动生成的图表。源代码仓库已整理至 GitHub: <https://github.com/Eurekaimer/statcomp-project>, 其中包含复现实验所需的代码、notebook、模拟数据和结果文件。

A.3 实验复现

完整实验复现步骤如下:

- (1) **环境准备**。在 RStudio 中打开 Project/ 目录, 依次执行:

```
install.packages("renv")
source("restore_environment.R")
source("R/load_project.R")
load_project()
```

其中 `restore_environment.R` 调用 `renv::restore()` 根据 `renv.lock` 安装所有依赖包的精确版本。

- (2) **初步运行检查**。执行 `source("scripts/smoke_test.R")`, 验证环境和主链路正常。
- (3) **主线实验**。执行 `source("run_all.R")` 依次运行全部主线实验 (自主实现验证、低维/中等规模比较、iterative 补充实验、样本量影响等)。各实验脚本以独立子进程运行, 单个实验失败不影响后续实验。

(4) **报告生成**。执行 `source("scripts/build_report_assets.R")` 将图表整理至报告目录，随后编译 LaTeX 源文件生成 PDF。

实验输出文件的对应关系如下：

- 表格输出以 CSV 格式保存于 `results/tables/`;
- 报告图像保存于 `results/figures/`;
- 报告中的主要数值和图像可由上述结果文件及作图脚本对应核对。